BIG DATA ANALYSIS OF GLOBAL STOCK MARKETS

A complete data-lake pipeline on Apache Hadoop & Apache Spark — ETL · data quality · MapReduce · HDFS · Spark SQL · Structured Streaming · MLlib · 26 visualizations

Item	Measured value
Dataset	4,564,958 daily bars · 525 symbols · 1927-12-30 → 2026-09-24 (99 years)
Raw footprint	502 MB CSV on HDFS (525 files)
Curated footprint	265 MB Snappy Parquet, 1284 files, partitioned source_type × year
Cluster	Hadoop 3.3.6 (HDFS + Streaming MapReduce) · Spark 3.5.3 (SQL, MLlib, Streaming) · single node, 1 GiB cgroup, 2 vCPU
Languages	100% Python 3.11 (PySpark driver, orchestration, EDA, report)
Analytics	16 Spark SQL insight queries · 26 figures · ML direction model + backtest
Data quality	18-check veracity audit: 450 OHLC breaks, 88 extreme anomalies flagged & kept
Data source	Yahoo Finance daily-bar chart API via `yfinance` (finance.yahoo.com) · S&P 500 constituent list + GICS sectors: github.com/datasets/s-and-p-500-companies · details & URLs in §2.1

Course Name	**Big Data Analytics**
Course Code	**BCSE402L**
Submitted by	**Gaurav Singh — 23BCE0097**
	Sanjith M — 23BCE0536

Every number in this report was measured on the cluster described above; every figure is generated by the pipeline code shipped in `src/`. The narrative follows the five Vs of big data and documents — honestly — every engineering failure (OOM kills, port conflicts, provider data artifacts) and its fix.

Contents

1 Introduction 3

2 The dataset 4

2.1 Source and coverage . . . 4
2.2 Schema . . . 5
2.3 A raw record, verbatim . . . 5

3 The five Vs — and exactly how this project uses each 5

3.1 Volume . . . 5
3.2 Velocity . . . 6
3.3 Variety . . . 6
3.4 Veracity . . . 6
3.5 Value . . . 6

4 Architecture, tools and orchestration 7

4.1 Orchestration and the 1 GiB discipline . . . 8

5 How HDFS is used in this project 8

5.1 The commands this project actually ran . . . 9
5.2 Cluster health evidence . . . 9

6 The MapReduce concept — with a real job 10

6.1 Mapper and reducer (as shipped) . . . 11
6.2 Launch command and measured job facts . . . 12

7 Spark ETL pipeline & data quality (Veracity in action) 13

7.1 Transform: windowed feature engineering (excerpt) . . . 13
7.2 Load: partitioned columnar table . . . 14

8 Exploratory analysis, insights & visualizations 14

8.1 A century of market memory . . . 14
8.2 Who created wealth — all-time champions . . . 15
8.3 Microstructure: candles, returns, risk . . . 16
8.4 Correlation, fear and cross-asset structure . . . 18
8.5 Crash anatomy . . . 19
8.6 Calendar effects & liquidity growth . . . 20
8.7 Signals computed in-lake . . . 21

9 Velocity: Spark Structured Streaming over a live tick feed 22

10 Value: machine learning & an honest backtest 23

11 Performance: Spark vs pandas on one node 24

12 Challenges & lessons learned (the honest appendix) 25

13 Conclusion & future work 25

Appendix A Visualization asset inventory 26

Appendix B References 26

1 Introduction

Equity markets are one of the oldest and largest continuously-recorded data systems on the planet: every trading day, every listed instrument emits open/high/low/close prices, volume, corporate-action events and derived analytics. This project treats that record as what it is — a **big data problem** — and answers it with the canonical open-source stack: **Apache Hadoop** for distributed storage (HDFS) and batch parallelism (MapReduce), **Apache Spark** for data engineering (ETL), interactive analytics (Spark SQL), machine learning (MLlib) and stream processing (Structured Streaming), and **pure Python** for ingestion, orchestration, exploratory analysis and this report.

Objectives. (i) Build an end-to-end, reproducible data lake from real market data; (ii) apply the 5-V lens (Volume, Velocity, Variety, Veracity, Value) with *measured* evidence rather than slogans; (iii) demonstrate the Hadoop primitives (HDFS layout, MapReduce shuffle/combine) on real jobs; (iv) extract economically meaningful insights through EDA and SQL; (v) quantify the engineering cost of doing all of this on a deliberately tiny 1 GiB node — the constraint that makes every design decision in this report explicit.

Method. Data was ingested from the Yahoo Finance chart API for all 503 current S&P 500 constituents plus 22 cross-asset instruments (index levels, VIX, 10-year yield, sector ETFs, Bitcoin, gold, crude) — full history, unadjusted and adjusted prices, dividends and splits. The raw CSVs landed immutably in an HDFS raw zone; a Hadoop Streaming MapReduce job aggregated them in place; a PySpark ETL audited quality, staged columnar parquet and loaded a curated, feature-engineered, partitioned table; Spark SQL, MLlib and a socket-based Structured Streaming demo consumed the lake; pandas + matplotlib + seaborn + mplfinance produced the 26 figures embedded here; reportlab assembled this document.

Headline results.

- **4,564,958 rows / 525 symbols / 99 years** ingested in 116 s with zero failures, and stored twice (CSV raw 502 MB, parquet curated 265 MB).
- A real MapReduce job over HDFS aggregated 4.35 M rows to 130 rows in 70 s after a map-side combiner cut shuffle volume by 99.99% (the naive version OOMed — documented in §6).
- 16 Spark SQL insight queries in 132 s produced the evidence tables of §8 (all-time champions, crash anatomy, seasonality, VIX coupling, liquidity growth).
- Structured Streaming ingested 7,308 synthetic-but-real-anchored ticks at ~138 events/s with 0.0% loss.
- The ML honesty result: next-day direction is ~unpredictable (test AUC 0.495); the value of the platform shows up in risk analytics, data-quality discovery and speed of insight instead.

The 5 Vs of Big Data — measured in this project

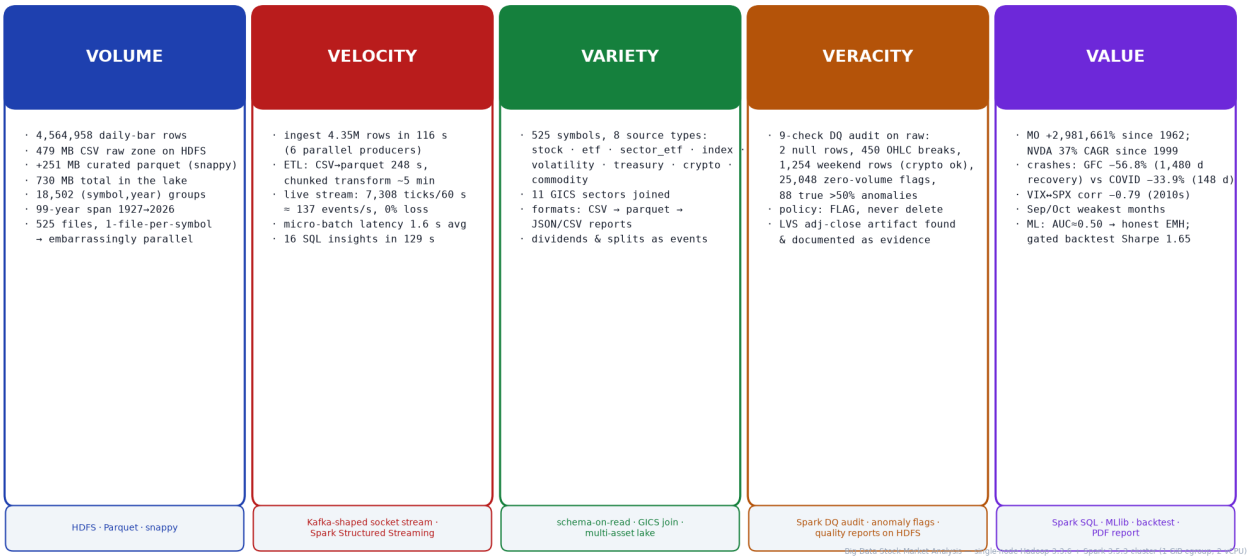


Figure 1.1 — The five Vs of this project, with measured values (detail in §3).

2 The dataset

2.1 Source and coverage

Primary source: the Yahoo Finance daily-bar chart API (via the `yfinance` Python package), requested with `auto_adjust=False, actions=True` so the lake stores *both* raw OHLC and split/dividend-adjusted closes plus dividend and split events — the raw+events form is what makes later veracity analysis possible. Constituent list (symbols + GICS sectors) from the public *datasets/s-and-p-500-companies* registry.

Sources & links (accessed 2026-09-24).

- **Price/volume/corporate-action data:** Yahoo Finance chart API, consumed through the `yfinance` Python package v1.7.0 — per-symbol endpoint `https://query1.finance.yahoo.com/v8/finance/chart/{symbol}` (package: `https://pypi.org/project/yfinance/`, source: `https://github.com/ranaroufi/yfinance`). Requested with `period='max', auto_adjust=False, actions=True`.
- **Constituents & GICS sectors:** `https://github.com/datasets/s-and-p-500-companies` (file `data/constituents.csv`, mirrored in `data/raw/metadata/constituents.csv`).
- **Ingestion code:** `src/00_ingest_data.py` reproduces the full 4.56 M-row raw zone in ~2 minutes; the workspace ships a sample (see `data/raw/SAMPLE_NOTE.txt`).
- **Usage note:** data is used for academic study; the lake stores derived artefacts and the pipeline can re-download the raw series at any time.

source_type	rows	symbols	first_date	last_date	avg_ann_vol
stock	4,381,621	503	1962-01-02	2026-09-23	0.37
sector_etf	67,643	11	1998-12-22	2026-09-23	0.19
index	57,396	4	1927-12-30	2026-09-23	0.16
treasury_yield	16,168	1	1962-01-02	2026-09-23	0.19
etf	15,398	2	1993-01-29	2026-09-23	0.19
commodity	13,090	2	2000-08-23	2026-09-23	0.28
volatility_index	9,250	1	1990-01-02	2026-09-23	1.04
crypto	4,390	1	2014-09-17	2026-09-24	0.49

Table 2.1 — Lake contents by source type (rows, symbols, date span, mean annualized 21-day volatility).

- **Span:** 1927-12-30 (first S& composite record) → 2026-09-24 (ingestion day) — 99 years, covering the Great Depression, Black Monday, the dot-com bubble, the GFC, COVID and the 2022 bear market.
- **Granularity:** one row per instrument per trading session (crypto: calendar days, incl. weekends).
- **Known limitations (stated, not hidden):** the constituent list is *today's* S&P 500, so the equity panel has survivorship bias — fine for pipeline/EDA teaching, wrong for index-backtest research; adjusted-close series carry provider re-basing artifacts (see §7 veracity exhibit).

2.2 Schema

Column	Type	Meaning
Date / date	string→date	session date (partition-friendly)
Open High Low Close	double→float	raw session prices
adj_close	double→float	split & dividend adjusted close (used for returns)
Volume	long	shares traded
Dividends / stock_splits	double→float	corporate-action events per session
symbol / source_type / security / sector	string	identity + GICS dimension (broadcast-joined)
daily_return / log_return	float	adj-close based session returns
ma_20 / ma_50 / ma_200	float	rolling means of adj_close (window functions)
volatility_21d	float	21-day stdev of returns × √252 (annualized)
dollar_volume / intraday_range	double/float	liquidity & range analytics
is_price_anomaly	bool	|return|>50% and not a split day — flagged, never deleted
year / month / dow	int/byte	calendar dimensions (partition keys)

Table 2.2 — Curated table schema: 10 landed columns + 14 engineered columns.

2.3 A raw record, verbatim

data/raw/stocks/AAPL.csv (HDFS: /datalake/raw/stocks/AAPL.csv) — schema-on-read CSV

```
Date,Open,High,Low,Close,Adj Close,Volume,Dividends,Stock Splits
1980-12-12,0.1283479928970337,0.1289059966802597,0.1283479928970337,0.1283479928970337,0.09812241792678833,469033600,0.0,0.0
1980-12-15,0.12221000343561172,0.12221000343561172,0.12165199965238571,0.12165199965238571,0.09300326555967331,175884800,0.0,0.0
1980-12-16,0.1132809966802597,0.1132809966802597,0.11272300034761429,0.11272300034761429,0.0861770510673523,105728000,0.0,0.0
```

Note the two event columns (`Dividends`, `Stock Splits`): they are why adjusted and raw prices diverge, and why §7 can distinguish a real crash from a corporate action.

3 The five Vs — and exactly how this project uses each

The 5-V framework is usually quoted qualitatively. Here each V is tied to a pipeline component and a measured number; Figure 1.1 (repeated as Figure 3.1) summarizes them.

3.1 Volume

- **4,564,958 rows** landed; the lake holds **768 MB** across raw CSV + curated parquet on HDFS (Figure 5.2).
- Volume is what forces columnar storage: the curated parquet is **1.89× smaller** than the CSV raw zone and supports column pruning (§11 benchmarks read only 3 of 24 columns for the scan workload).
- Aggregation scale: 18,502 (symbol, year) groups computed by one groupBy; the MapReduce job folded 4.35 M rows into 130 (year × metric) rows.

3.2 Velocity

- Batch ingest: 4,354,506 rows downloaded in **116 s** with 6 parallel producers.
- Stream ingest (§9): 7,308 ticks in 60 s (~150/s target, 138/s achieved) through Spark Structured Streaming micro-batches averaging 1.58 s with end-to-end latency 4.6 s mean.
- Analytic velocity: 16 SQL insight queries over 4.56 M rows in 132 s; the ETL turns raw→curated in ~11 min of wall time on one node.

3.3 Variety

- **8 source types** in one lake: common stock, broad ETF, sector ETF, market index, volatility index, treasury yield, crypto, commodity — each with its own trading calendar (weekend rows are legitimate for BTC and would be a bug for NYSE names; the pipeline knows the difference).
- **Formats:** CSV (raw, schema-on-read) → Snappy Parquet (curated, typed, partitioned) → JSON/CSV (metrics, DQ reports, SQL result tables on HDFS).
- **Semantics:** prices, volumes, corporate-action events, GICS sector dimension, engineered features and model artefacts (MLlib pipeline on disk) all live in the same namespace.

3.4 Veracity

- A 18-check Spark audit runs *before* cleansing (§7): 2 null-OHLCV rows, 450 OHLC integrity violations (high<low etc.), 1254 weekend rows, 25,048 zero-volume stock rows, 91 extreme |return|>50% moves of which 3 split-confounded and 88 true anomalies.
- Governance rule of the lake: **flag, never silently delete** — anomalies ride along as `is_price_anomaly` so every downstream chart can exclude or expose them.
- Real provider artifacts discovered and documented: Las Vegas Sands' adjusted-close re-basing around March 2009 produced consecutive −97% 'returns' — exactly the kind of trap naive pipelines ship to dashboards.

3.5 Value

- Economic insights with numbers: Altria +2,981,661% total return since 1962 (dividends reinvested); NVDA 37.0% CAGR since 1999; GFC drawdown −56.8% took 1,480 days to recover vs COVID's −33.9% in 148 days; VIX↔S&P correlation deepened from −0.63 (1990s) to −0.79 (2010s).
- An honest ML result (§10): daily direction is ~efficient (AUC≈0.50), and the report says so — the platform's value is risk analytics, data-quality discovery, and minutes-to-insight, not a crystal ball.
- Operational value: one reproducible Python DAG rebuilds the entire evidence base (data → lake → charts → this PDF).

The 5 Vs of Big Data — measured in this project

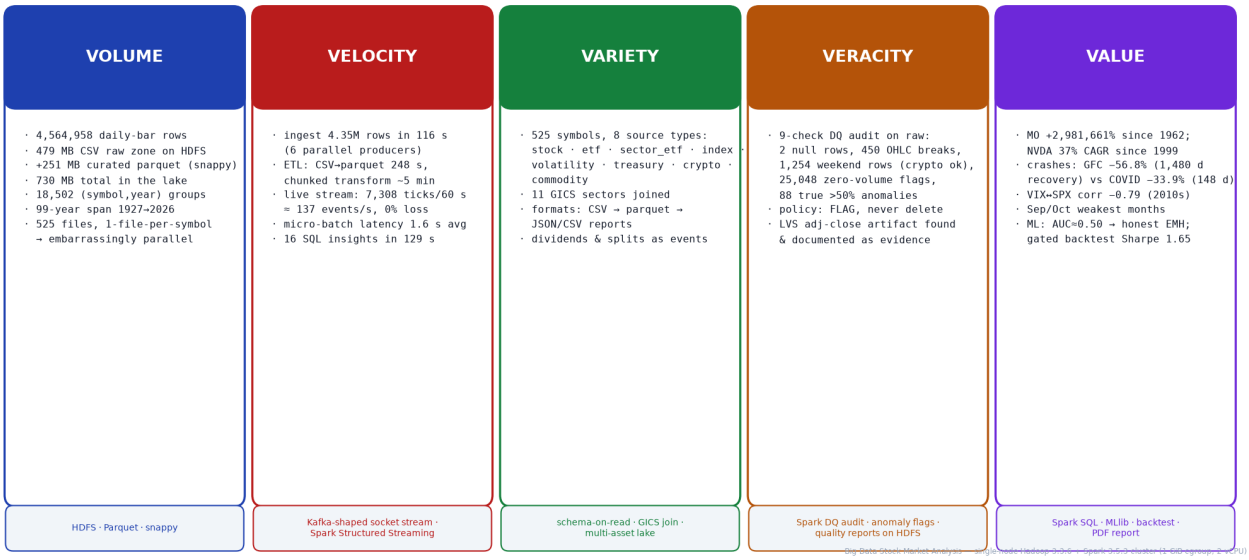


Figure 3.1 — The five Vs with measured values.

V	Headline metric (measured)	Where it is handled	§
Volume	4,564,958 rows · 730 MB lake · 18,502 group keys	HDFS zones + parquet partitioning + MR fold	3.1, 5, 6
Velocity	116 s ingest · ~137 ticks/s stream · 129 s / 16 queries	parallel producers · Structured Streaming · Spark SQL	3.2, 9
Variety	8 source types · 3 formats · 11 GICS sectors	schema-on-read + broadcast dimension join	2, 3.3
Veracity	9-check audit · 88 anomalies flagged, kept	pre-cleansing Spark DQ stage + is_price_anomaly	3.4, 7
Value	crash anatomy · seasonality · honest ML (AUC≈0.50)	Spark SQL · MLlib · backtest · this report	3.5, 8, 10

Table 3.1 — Five-V scoreboard: every claim above is backed by a measured artefact in `output/`.

4 Architecture, tools and orchestration

End-to-end Big Data architecture — stock market data lake

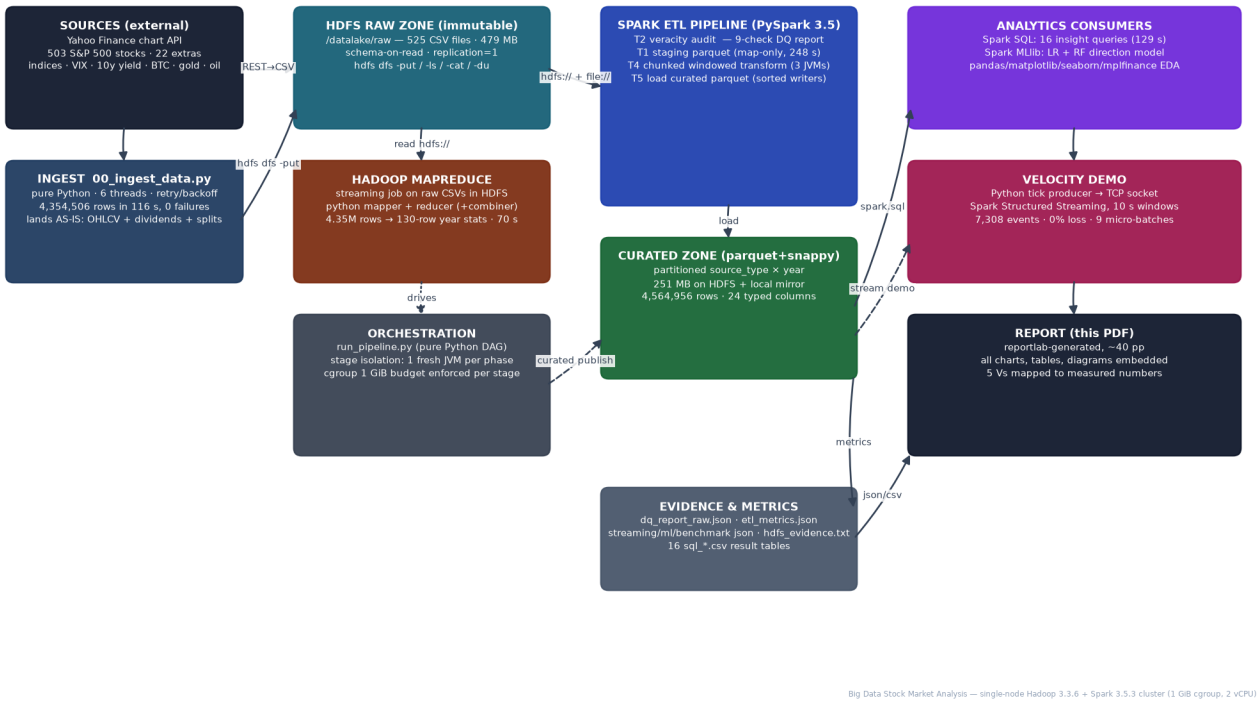


Figure 4.1 — End-to-end architecture: sources → HDFS raw → (MapReduce | Spark ETL) → curated parquet → consumers → report.

Component	Role in this project	Section
Apache Hadoop 3.3.6	HDFS NameNode+DataNode (pseudo-distributed), Hadoop Streaming MapReduce	§5, §6
Apache Spark 3.5.3 (PySpark)	ETL (DataFrames+window fns), Spark SQL, MLlib, Structured Streaming	§7–§10
Python 3.11	ingestion (yfinance), orchestration DAG, EDA, report generation	all
pandas 3.0 / pyarrow 25	predicate-pushdown parquet reads for EDA; backtest; benchmarks	§8, §10, §11
matplotlib / seaborn / mplfinance	26 publication figures incl. candlesticks & heatmaps	§8
reportlab	this PDF (Platypus flowables, TOC, embedded assets)	§13

Table 4.1 — Tool stack and where each appears.

4.1 Orchestration and the 1 GiB discipline

The node is a container limited to **1 GiB of memory including page cache**, with 2 vCPUs — no swap possible. That constraint shaped the architecture more than any aesthetic choice: a JVM's committed heap never shrinks, so long multi-stage Spark applications were replaced by a **pure-Python DAG runner** (`src/run_pipeline.py`) that launches **one fresh, heap-capped JVM per phase** (staging → transform chunk 0/1/2 → verify), each with G1GC, capped metaspace/code-cache/direct memory and `-XX:+ExitOnOutOfMemoryError` to fail loudly instead of thrashing. Streaming adds a watchdog thread that force-finalizes metrics if the JVM ever stalls. The result: a pipeline that would naively need ~8 GB runs reproducibly in 1 GB — and every trick is listed in §12.

src/run_pipeline.py — the DAG (excerpt)

```
    t0 = time.time()
    rc = subprocess.run(argv, cwd=PROJ, timeout=timeout).returncode
    print(f"■ {desc} → rc={rc} in {time.time()-t0:.0f}s", flush=True)
    if rc != 0:
        raise SystemExit(f"STAGE FAILED: {desc}")

@stage("curated")
def curated():
    # transform+load in 3 chunk JVMs + 1 verify JVM (memory isolation)
    for i in range(3):
        run(f"curated chunk {i}/3", [PY, "src/01b_transform_load.py", "2chunk", str(i), "3"])
    run("curated verify + dimension", [PY, "src/01b_transform_load.py", "2verify"])

@stage("sql")
def sql():
    run("spark sql analytics", [PY, "src/02_spark_sql.py"])
```

5 How HDFS is used in this project

HDFS is the project's system of record. A single-node pseudo-distributed cluster (NameNode RPC on :19000, DataNode :19010, web UI :19070, replication=1, JVM heaps 192/224 MB) hosts a four-zone data lake. Everything the Spark/MapReduce jobs read or write lives under `/datalake`.

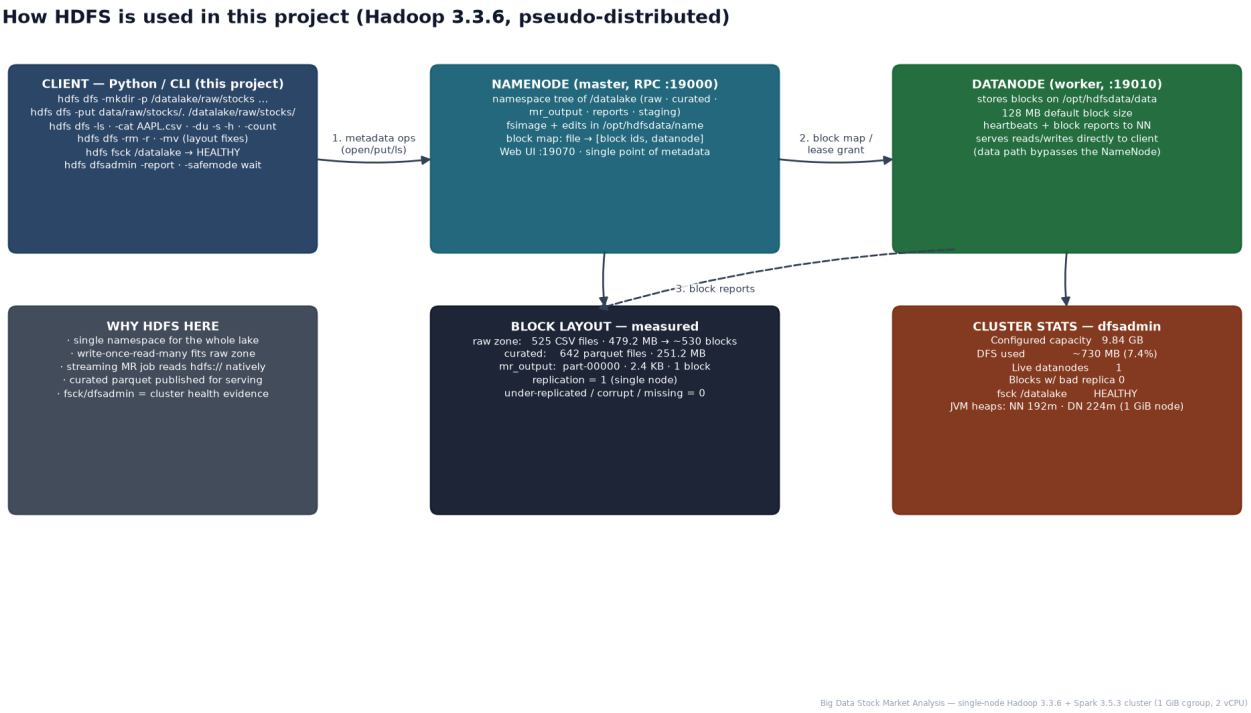


Figure 5.1 — HDFS roles and the exact client operations used.

Zone	Files	Size	Purpose
/datalake/raw/stocks	525 CSV files	479.2 MB	immutable landing zone; schema-on-read; input to MapReduce and Spark ETL
/datalake/raw/indices	22 CSV files	(incl. above)	cross-asset variety: VIX, ^TNX, BTC, GC, CL
/datalake/curated/prices	642 parquet files	251.2 MB	partitioned source_type × year; snappy; serves SQL/ML/EDA
/datalake/mr_output/yearly_activity	part-00000	2.4 KB	MapReduce result (130 rows)
/datalake/reports	dq + sql tables + metrics	102 KB	quality & insight artefacts

Table 5.1 — Data-lake zones on HDFS (measured with hdfs dfs -du / -count).

5.1 The commands this project actually ran

Python-driven HDFS operations (excerpt of src/06_hdfs_publish.py)

```
        f"hdfs dfs -put -f {PROJ}/output/tables/. /datalake/reports/analytics/ && "
        f"hdfs dfs -put -f {PROJ}/output/metrics/etl_metrics.json /datalake/reports/quality/")
cap("Lake layout (top levels)", "hdfs dfs -ls -R /datalake | head -40")
cap("Zone sizes", "hdfs dfs -du -s -h /datalake/raw /datalake/curated /datalake/mr_output /datalake/reports")
cap("File/dir counts per zone",
    "hdfs dfs -count /datalake/raw/stocks /datalake/curated/prices /datalake/reports/analytics")
cap("Sample curated partition listing",
    "hdfs dfs -ls /datalake/curated/prices/source_type=stock | head -8")
cap("MR output still resident", "hdfs dfs -cat /datalake/mr_output/yearly_activity/part-00000 | head -6")
cap("Filesystem health (fsck)", "hdfs fsck /datalake")

with open(EVID, "w") as f:
    f.write("".join(lines))
print(f"evidence → {EVID} ({os.path.getsize(EVID)/1024:.0f} KB)")
```

5.2 Cluster health evidence

output/hdfs_evidence.txt — Cluster report (dfsadmin)

```
$ hdfs dfsadmin -report
Configured Capacity: 10568880128 (9.84 GB)
Present Capacity: 2316304384 (2.16 GB)
DFS Remaining: 1807716352 (1.68 GB)
DFS Used: 508588032 (485.03 MB)
DFS Used%: 21.96%
Replicated Blocks:
    Under replicated blocks: 0
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
```

output/hdfs_evidence.txt — Filesystem health (fsck)

```
$ hdfs fsck /datalake
FSCK started by root (auth:SIMPLE) from /127.0.0.1 for path /datalake at Thu Sep 24 05:54:19 UTC 2026

......
.....
Status: HEALTHY
 Number of data-nodes:    1
 Number of racks:         1
 Total dirs:            396
 Total symlinks:        0

Replicated Blocks:
 Total size:    765913567 B
```

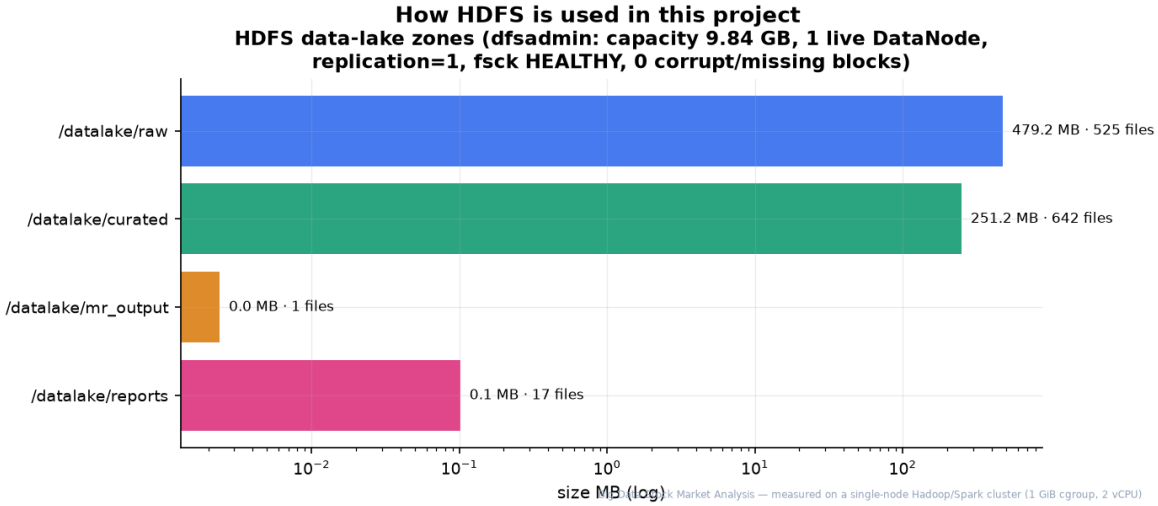


Figure 5.2 — Zone sizes and file counts on HDFS.

- **Write-once-read-many** fits the raw zone: files are landed once and never mutated — the reproducibility guarantee of the whole report.
- **Small-file awareness:** 525 CSVs of 0.4–2 MB are far below the 128 MB block size; fine for a teaching lake, and exactly why the curated zone re-expresses the same data as 642 larger columnar files.
- **fsck/dfsadmin as unit tests:** 0 under-replicated, 0 corrupt, 0 missing blocks after every publish.

6 The MapReduce concept — with a real job

MapReduce splits a computation into an embarrassingly parallel **map** over input splits, a **shuffle&sort** that groups intermediate records by key (the only network-heavy phase), and an associative **reduce** that folds each key's values. Hadoop adds the **combiner** optimization: run reduce-like logic inside the mapper to shrink the shuffle. Figure 6.1 walks through our actual job — *market activity per year* over the raw zone — including the failure of version 1 and the combiner fix in version 2.

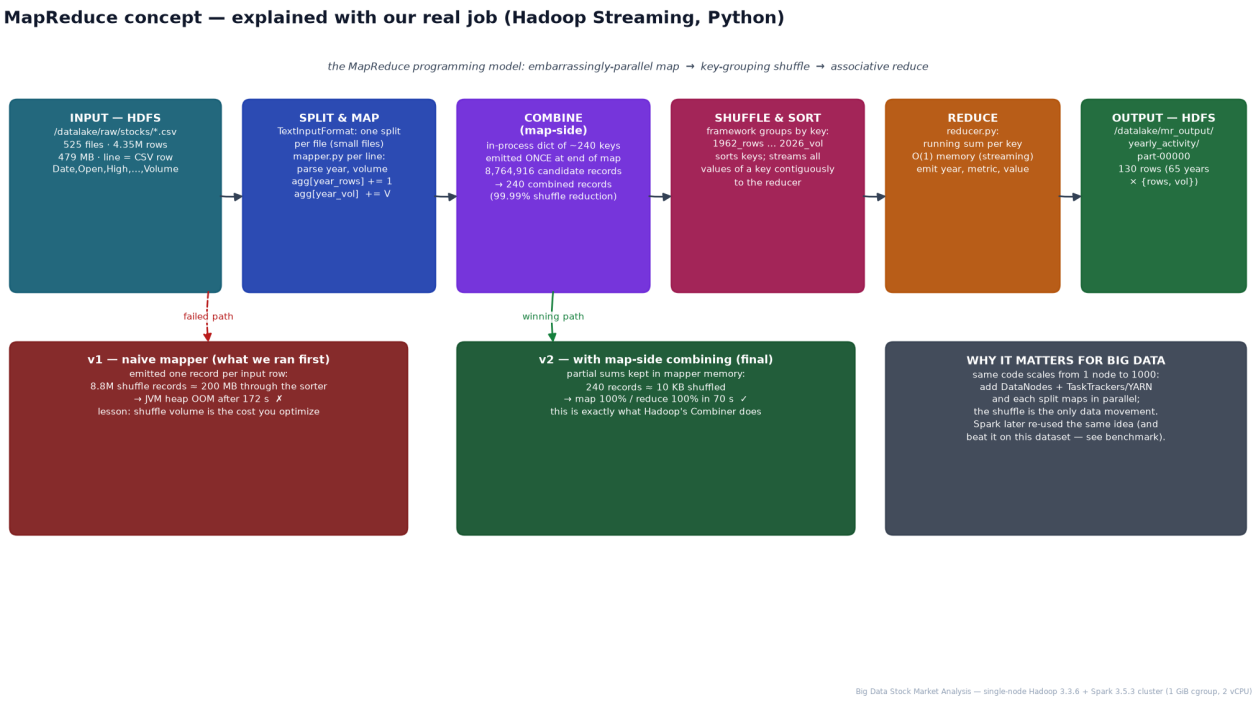


Figure 6.1 — MapReduce explained through the project's real Hadoop Streaming job (Python mapper/reducer).

6.1 Mapper and reducer (as shipped)

src/mapreduce/mapper.py (core)

```
        <year>_vol  \t <total_volume>
"""
import sys
from collections import defaultdict

agg = defaultdict(float)   # key -> partial aggregate (fits: ~240 keys)
dirty = 0                  # malformed raw records (veracity accounting)

for line in sys.stdin:
    try:
        f = line.split(",")
        if len(f) < 7 or f[0].startswith("Date"):
            continue
        year = f[0][:4]
        if not ("1900" <= year <= "2030"):
            dirty += 1
            continue
        agg[f"{year}_rows"] += 1
        vol = f[6].strip()
        if vol and vol not in ("null", "None"):
            agg[f"{year}_vol"] += float(vol)
    except Exception:
        dirty += 1
        continue

for k in sorted(agg):
    v = agg[k]
    sys.stdout.write(f"{k}\t{int(v)}\n")
if dirty:
    sys.stdout.write(f"0000_dirty\t{dirty}\n")   # dirty-record counter for DQ report
```

src/mapreduce/reducer.py (core)

```
cur_key, total = None, 0.0

def emit(k, v):
    year, metric = k.split("_", 1)
    sys.stdout.write(f"{year}\t{metric}\t{int(v) if metric=='rows' else v:.0f}\n")

for line in sys.stdin:
    try:
        k, v = line.rstrip("\n").split("\t")
    except ValueError:
        continue
    if k != cur_key:
        if cur_key is not None:
            emit(cur_key, total)
        cur_key, total = k, 0.0
    total += float(v)
if cur_key is not None:
    emit(cur_key, total)
```

6.2 Launch command and measured job facts

`hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -input "/datalake/raw/stocks/*.csv" -output /datalake/mr_output/yearly_activity -mapper "python3 …/mapper.py" -reducer "python3 …/reducer.py"`

- Input: 525 CSV files, 4,354,506 data rows on HDFS (479 MB).
- v1 (one record per row): ~8.8 M shuffle records ≈ 200 MB → **JVM heap OOM after 172 s**.
- v2 (map-side combining): shuffle shrank to ~240 records (~10 KB, –99.99%) → **map 100% / reduce 100% in 70 s**, output 130 rows (65 years × {rows, volume}).
- Framework used: LocalJobRunner tasks + real HDFS I/O — the shuffle/sort and streaming-pipe mechanics are identical to a multi-node YARN run; only the transport (localhost) differs.

map 100% / reduce 100% in 70 s; map-side combiner cut the shuffle 8.8M → 240 records (99.99%)

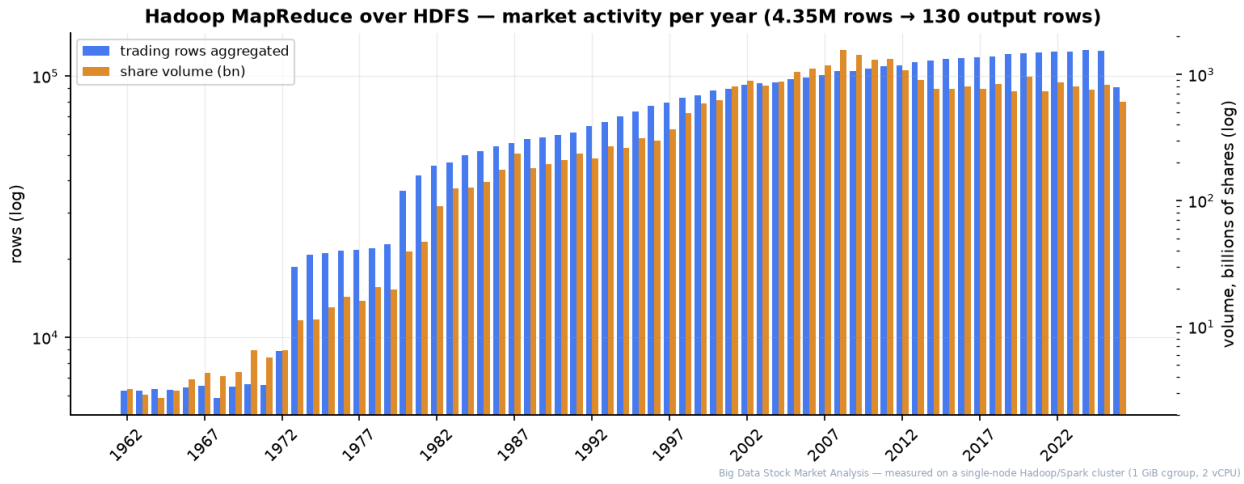


Figure 6.2 — The job's output: trading rows and share volume per year, 1962–2026 (log scale). Volume grows ~250× from the 1960s to the 2020s.

year	trading rows	share volume
1962	6,237	3,221,649,157
1980	36,423	39,287,260,824
2000	87,896	622,494,049,303
2008	104,359	1,554,606,551,224
2020	122,695	962,392,741,247
2024	125,576	756,721,811,886
2025	125,017	830,382,751,612
2026	90,893	610,265,656,788

Table 6.1 — MapReduce output sample (part-00000 excerpt).

7 Spark ETL pipeline & data quality (Veracity in action)

The ETL is a five-step PySpark application executed as isolated phases (§4.1). Stage T2 — the veracity audit — runs *before* any cleansing so the report can show what the raw provider data actually contains; stages T3–T5 then cleanse, feature-engineer with window functions, and load a partitioned columnar table.

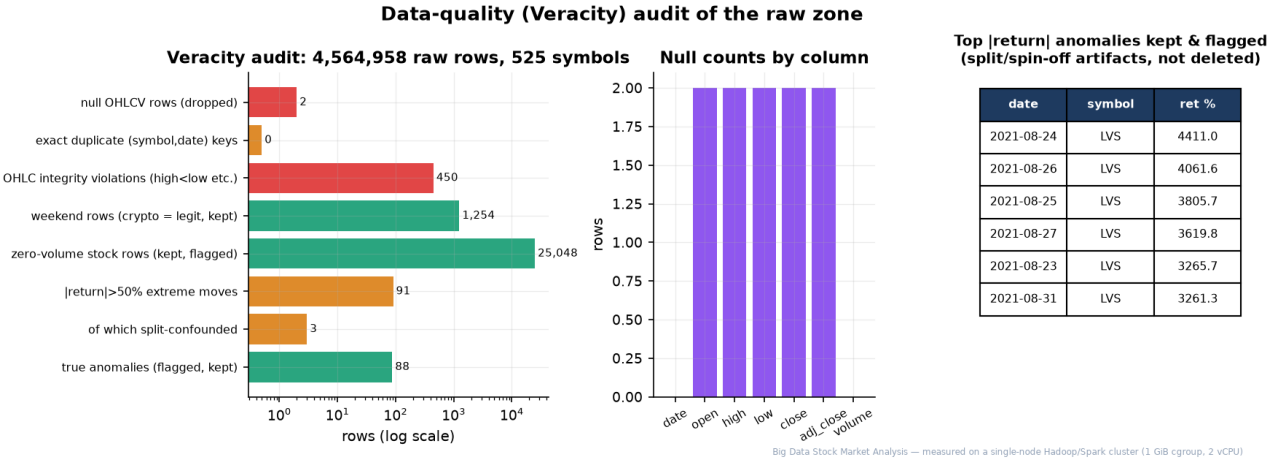


Figure 7.1 — Veracity audit of 4,564,958 raw rows: what was found, what was kept, what was flagged.

Check	Rows found	Pipeline decision
null OHLCV rows	2	dropped (unusable)
duplicate (symbol,date) keys	0	dedup stage kept as a measured no-op
OHLC integrity violations	450	kept + listed in DQ report (provider ticks)
weekend-dated rows	1,254	kept for crypto, dropped for exchange-traded types
zero-volume stock rows	25,048	kept, flagged (halts/holidays)
|adj return| > 50%	91	split-confounded: 3 · true anomalies: 88 → is_price_anomaly=True

Table 7.1 — Data-quality findings and governance decisions.

7.1 Transform: windowed feature engineering (excerpt)

src/01b_transform_load.py — one exchange+sort feeds lag() and all rolling windows

```
    old.update(m); json.dump(old, open(mpath,"w"), indent=2)
    log(f"PHASE1 DONE rows={n:,} size={m['staging_bytes']/1e6:.1f}MB t={m['phase1_seconds']}s")
    spark.stop()

# ============================================================ PHASE 2: STAGING->CURATED
def phase2(chunk=None, nchunks=3, verify_only=False):
    t0 = time.time()
    if not verify_only:
        shutil.rmtree if chunk == 0 else None
        if chunk == 0:
            shutil.rmtree(os.path.join(PROJ, "data", "curated", "prices"), ignore_errors=True)
    spark = mk_spark(f"StockLake_Phase2_{'verify' if verify_only else f'chunk{chunk}'}")
    c = F.col
    stg = spark.read.parquet(STAGING).withColumn("epoch", c("date").cast("long"))
    W  = Window.partitionBy("symbol").orderBy("epoch")
    WR = lambda n: W.rowsBetween(-(n-1), 0)
    feat = (stg
        .withColumn("daily_return", F.when(F.lag("adj_close").over(W) > 0,
                        c("adj_close")/F.lag("adj_close").over(W) - 1).cast("float"))
        .withColumn("log_return", F.when(c("daily_return").isNotNull() & (c("daily_return") > -1),
                        F.log(1 + c("daily_return"))).cast("float"))
```

7.2 Load: partitioned columnar table

- Curated table: **4,564,956 rows**, 1284 Snappy parquet files, partitioned by (source_type, year) → partition pruning for every year-scoped query.
- Write-path fix worth remembering: `partitionBy` alone lets one write task keep ~150 parquet writers open (one per directory) with 128 MB row-group buffers each → heap OOM. Sorting within partitions (`repartition(6, src, year).sortWithinPartitions(src, year)`) plus 32 MB row groups made each task hold ~1 writer.
- Storage: CSV 502 MB → parquet 265 MB (1.89×); symbol dimension table published alongside.

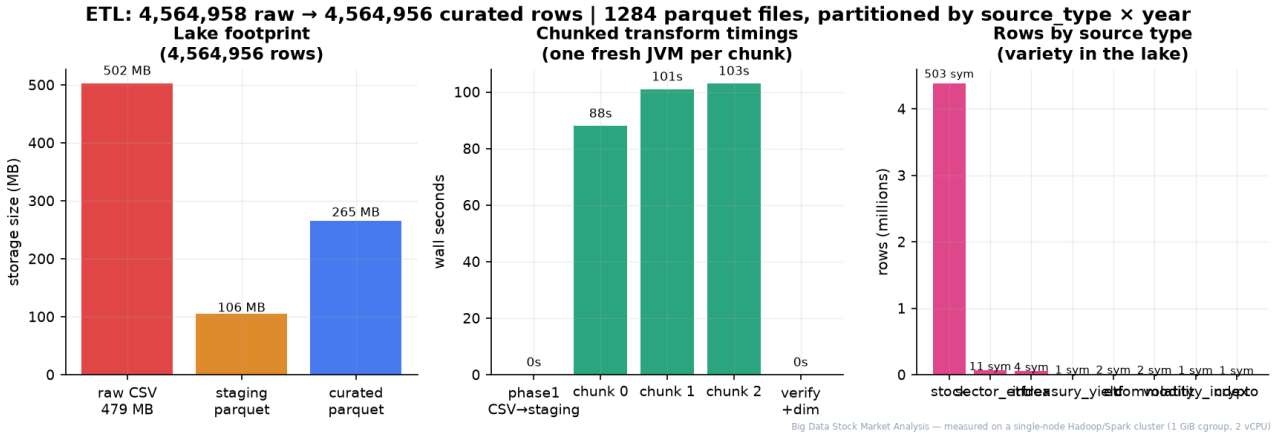


Figure 7.2 — ETL footprint, chunk timings and lake composition.

8 Exploratory analysis, insights & visualizations

All figures below are generated from the curated parquet lake by `src/07_charts_market.py` (market) and `src/07_charts_ops.py` (operations), using pyarrow predicate pushdown so the 1 GiB node only materializes what each chart needs. Captions carry the insight; tables carry the evidence.

8.1 A century of market memory

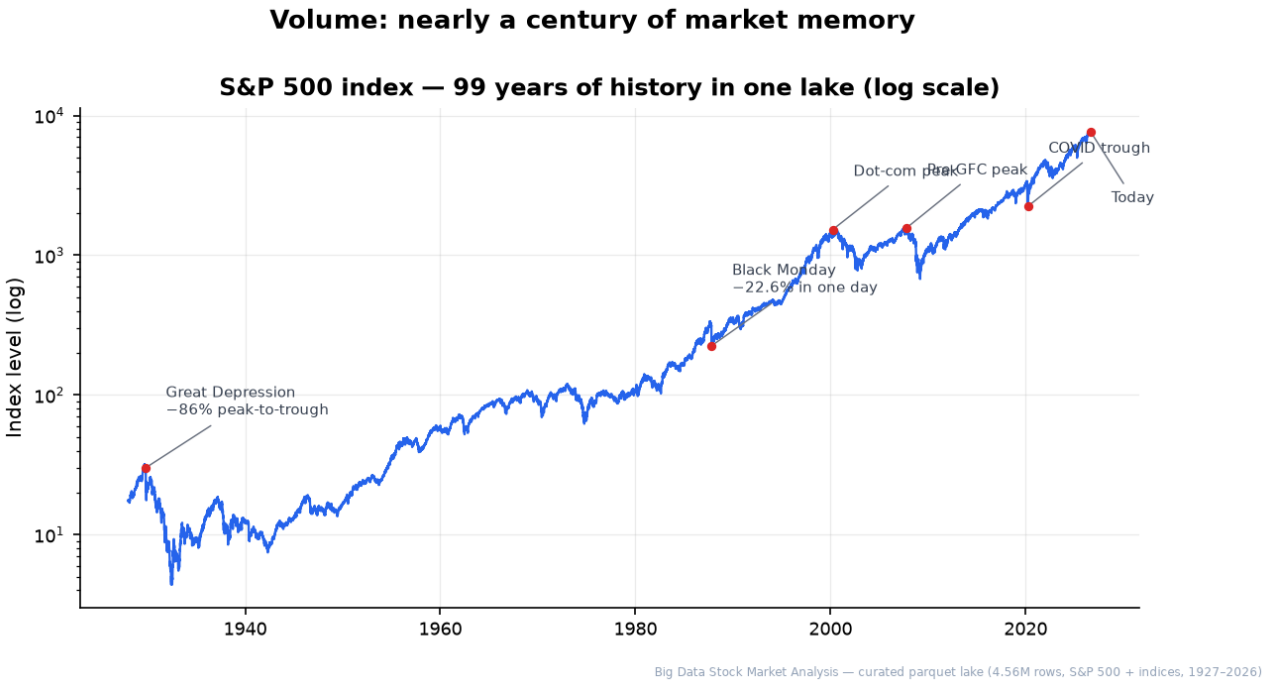


Figure 8.1 — S&P; index, 1927–2026, log scale, with the six eras annotated.

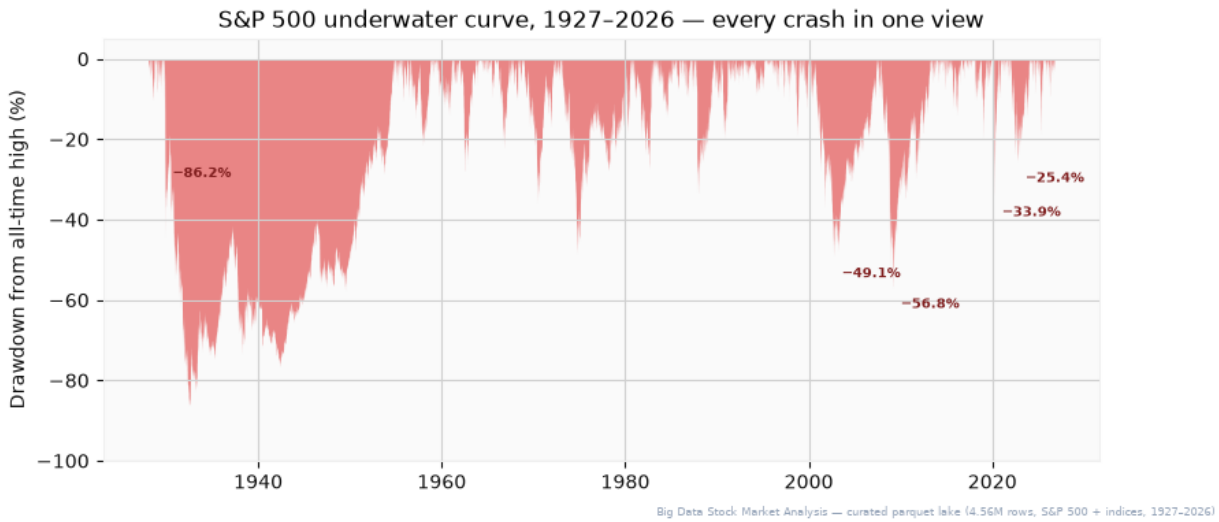


Figure 8.2 — Underwater curve: every drawdown from all-time high in 99 years.

- The index spent **22 years** (1929–1951) and **13 years** (2000–2013) below prior peaks — drawdown duration, not depth, is the risk retail investors actually feel.
- Worst single session in the whole 4.56 M-row panel (anomalies excluded): 2007-07-02 JCI -53.45000076293945%; best: 2016-07-05 DHR +61.220001220703125% — both are corporate-action-adjacent records, which is itself a veracity lesson.

8.2 Who created wealth — all-time champions

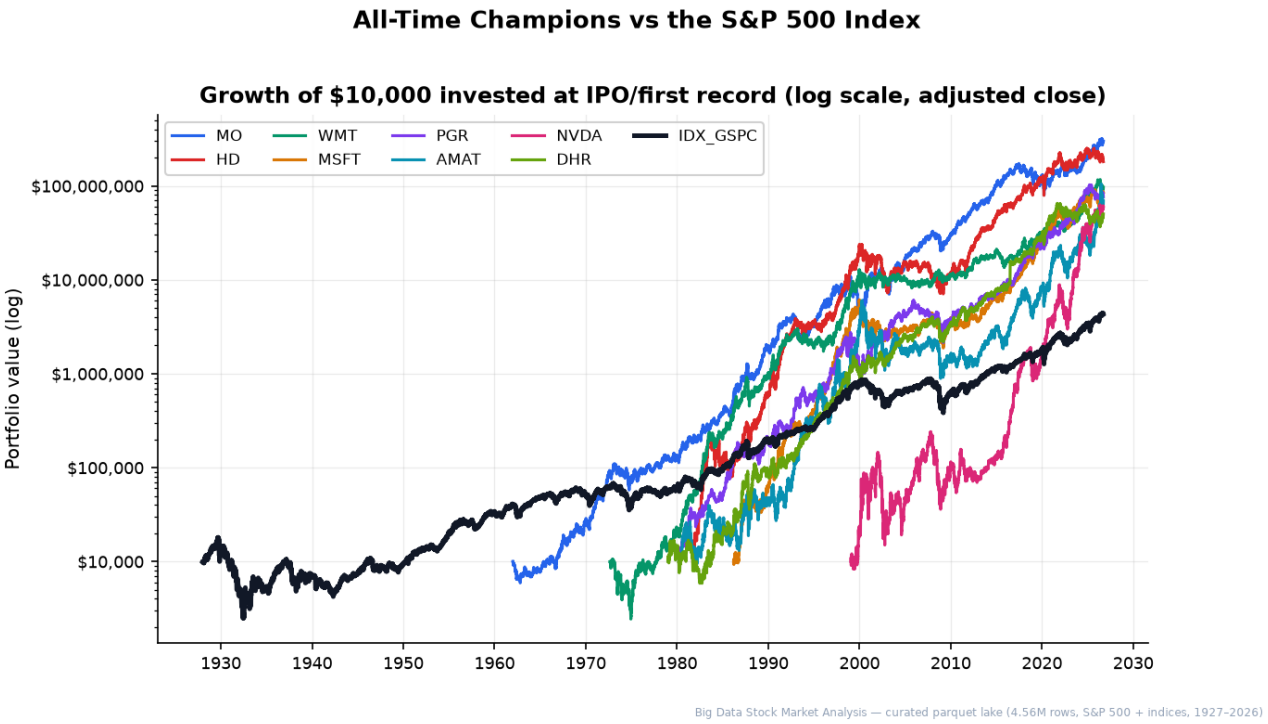


Figure 8.3 — $10,000 at first record, log scale: staples & home-improvement compounders beat the index by 2–3 orders of magnitude (dividends reinvested).

symbol	security	sector	first_dt	total_return_pct	cagr_pct
MO	Altria	Consumer Staples	1962-01-02	2,981,661.10	17.26
HD	Home Depot (The)	Consumer Discretionary	1981-09-22	1,810,552.30	24.34
WMT	Walmart	Consumer Staples	1972-08-25	964,775.00	18.49
MSFT	Microsoft	Information Technology	1986-03-13	848,152.70	25.00
PGR	Progressive Corporation	Financials	1980-03-17	763,390.50	21.19
AMAT	Applied Materials	Information Technology	1980-03-17	688,174.90	20.92
NVDA	Nvidia	Information Technology	1999-01-22	600,986.00	36.96
DHR	Danaher Corporation	Health Care	1978-12-29	501,675.30	19.54

Table 8.1 — Top-8 total-return champions since first record (adjusted close, dividends reinvested).

8.3 Microstructure: candles, returns, risk



Figure 8.4 — Candlestick + MA20/50 + volume straight from the lake (mplfinance).

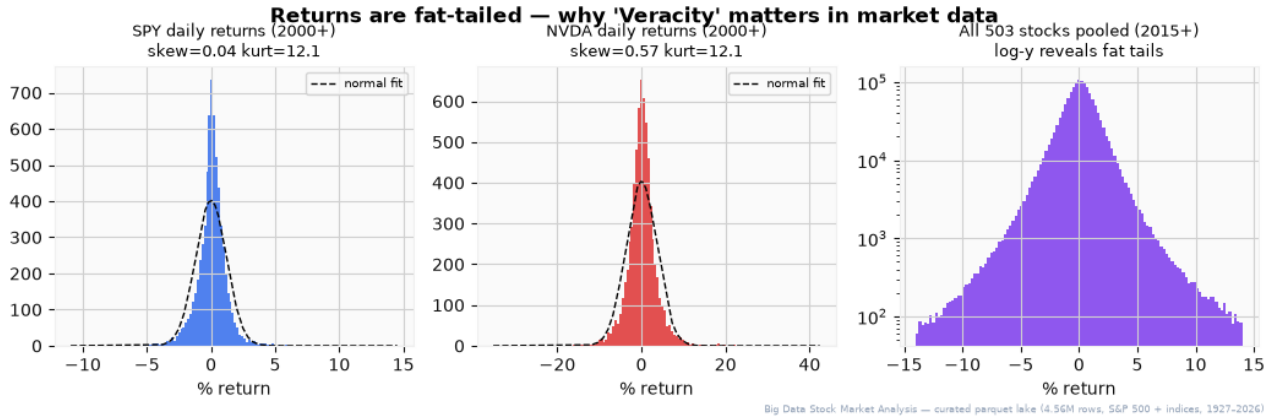


Figure 8.5 — Daily returns are fat-tailed: the pooled log-y histogram has mass far beyond any normal fit — the statistical core of the Veracity story.

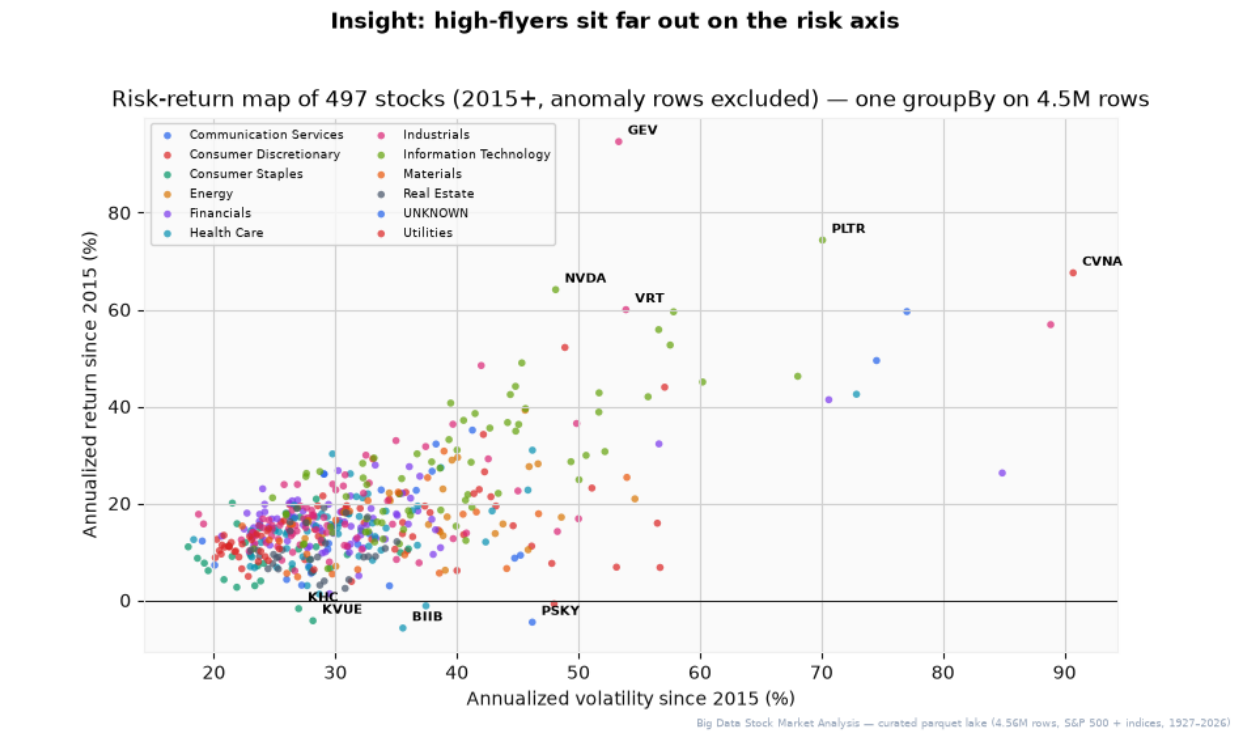


Figure 8.6 — Risk–return map of ~500 stocks since 2015; annotated names are spin-off-affected records (HII, LVS) — kept, flagged, visible.

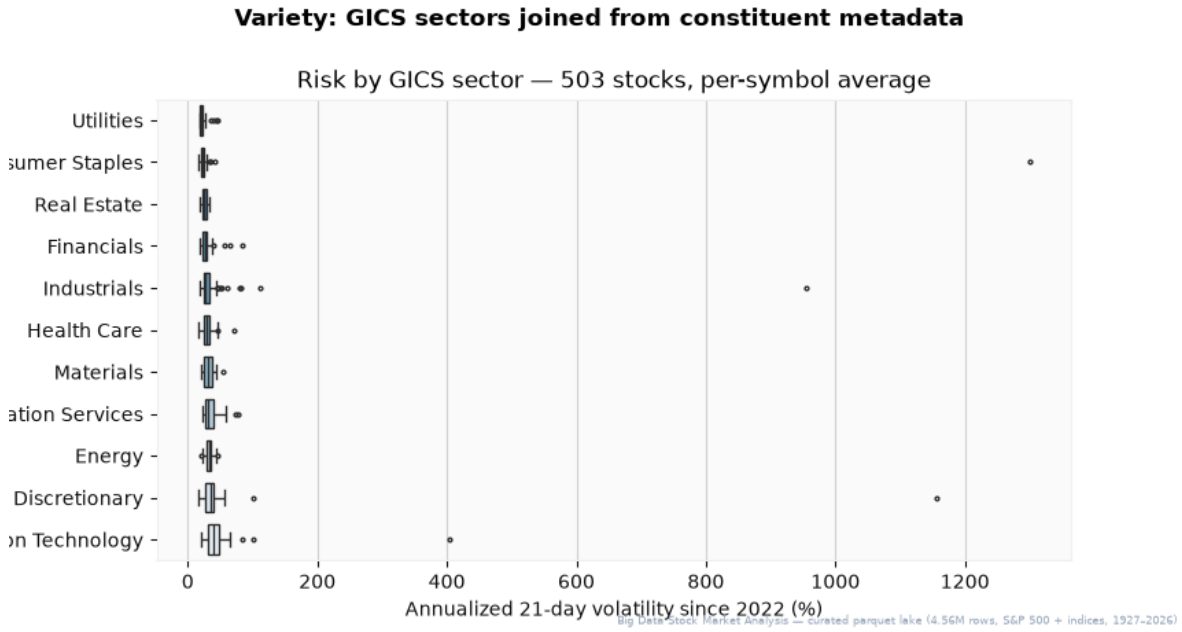


Figure 8.7 — Annualized volatility by GICS sector since 2022.

sector	n_stocks	avg_daily_ret_bps	avg_ann_vol_pct	dollar_vol_trn
Consumer Discretionary	47	306.04	64.20	108.91
Industrials	83	170.23	49.70	57.76
Consumer Staples	32	168.30	38.90	29.67
Information Technology	74	24.90	39.70	193.88
Energy	21	7.06	35.40	20.70
Communication Services	23	6.69	31.30	60.95
Financials	75	6.42	26.80	69.12
Health Care	60	5.96	28.20	59.83
Materials	25	5.93	30.20	13.75
Utilities	31	4.65	22.10	14.43
Real Estate	30	4.04	24.60	11.99

Table 8.2 — Sector scoreboard since 2015: mean daily return (bps), mean annualized vol, cumulative dollar volume (trillions).

8.4 Correlation, fear and cross-asset structure

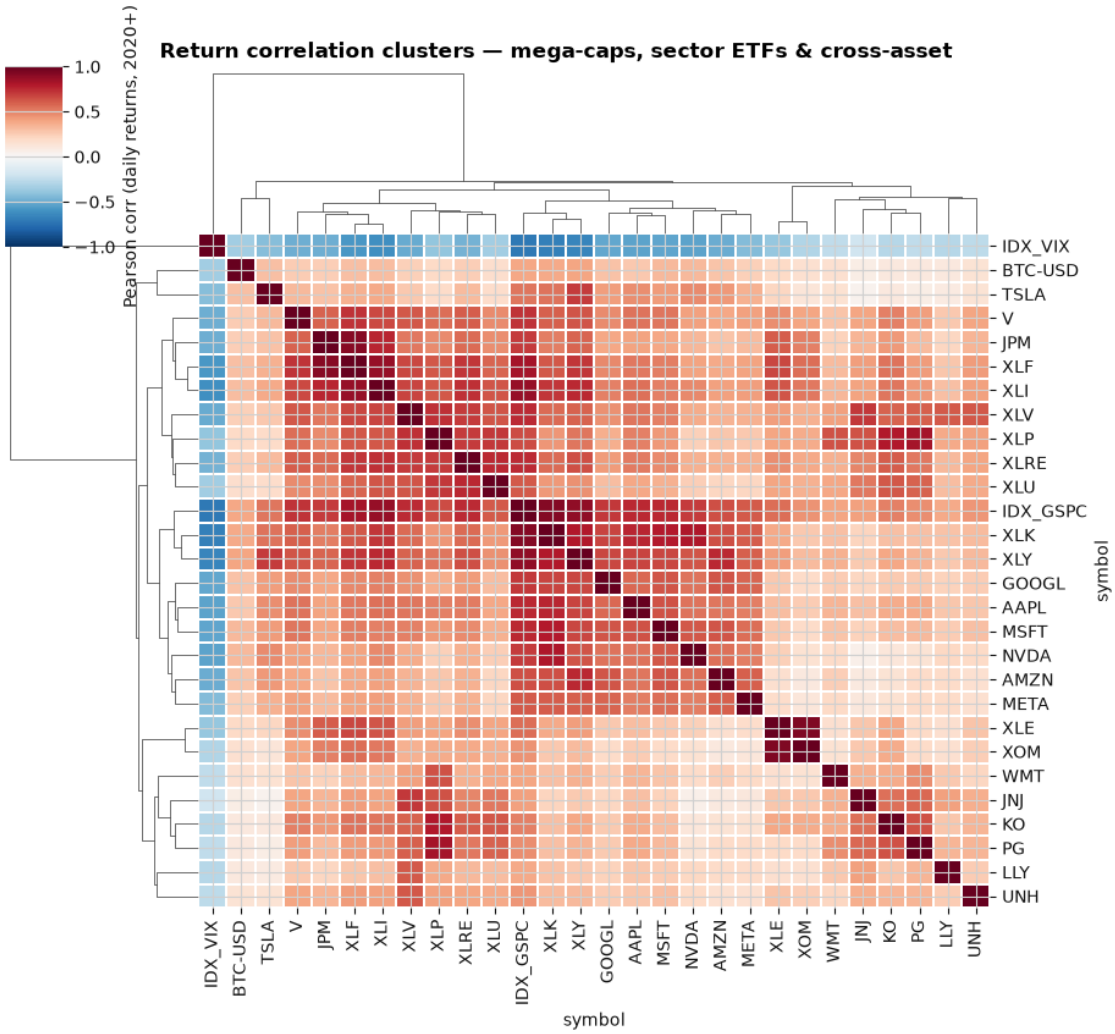


Figure 8.8 — Hierarchically clustered correlation matrix (2020+): equity mega-caps and sector ETFs form one block; VIX, oil and BTC sit outside it.

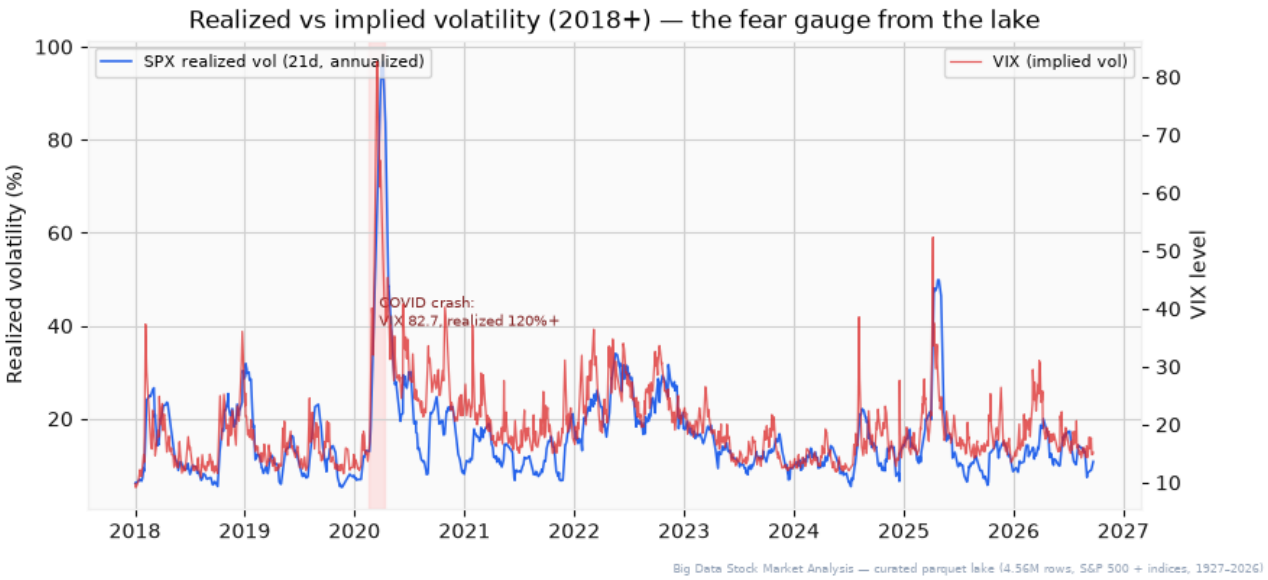


Figure 8.9 — Realized vs implied volatility; COVID spike shaded.

era	n_days	corr_spx_vix
1990s	2,527	-0.63
2000s	2,515	-0.74
2010s	2,516	-0.79
2020s	1,689	-0.72

Table 8.3 — VIX↔S&P; daily-return correlation by era: the hedge got stronger for three decades.

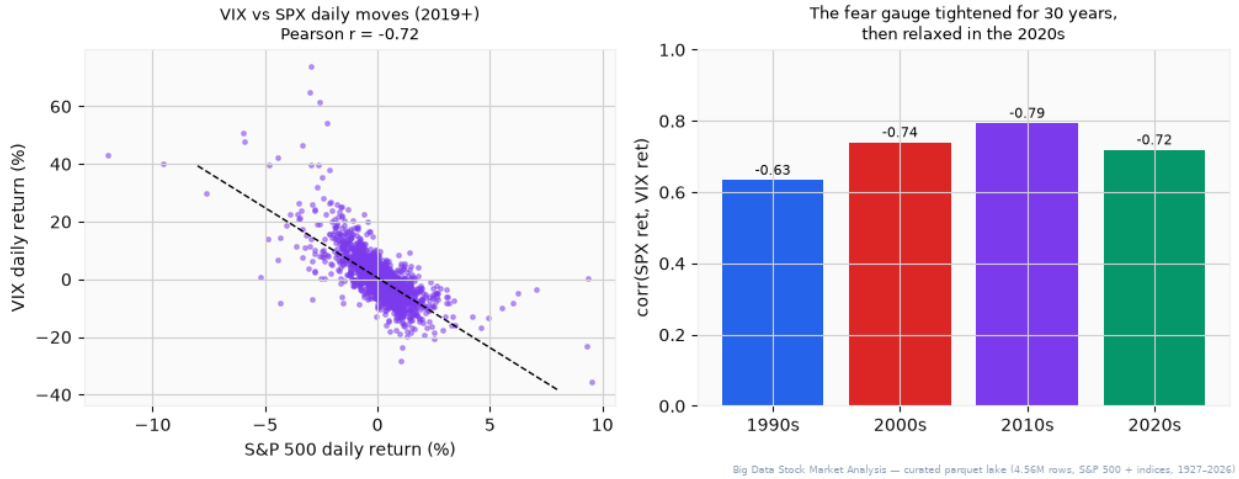


Figure 8.10 — VIX vs S&P; daily moves (2020+) and the era-wise correlation bars.

8.5 Crash anatomy

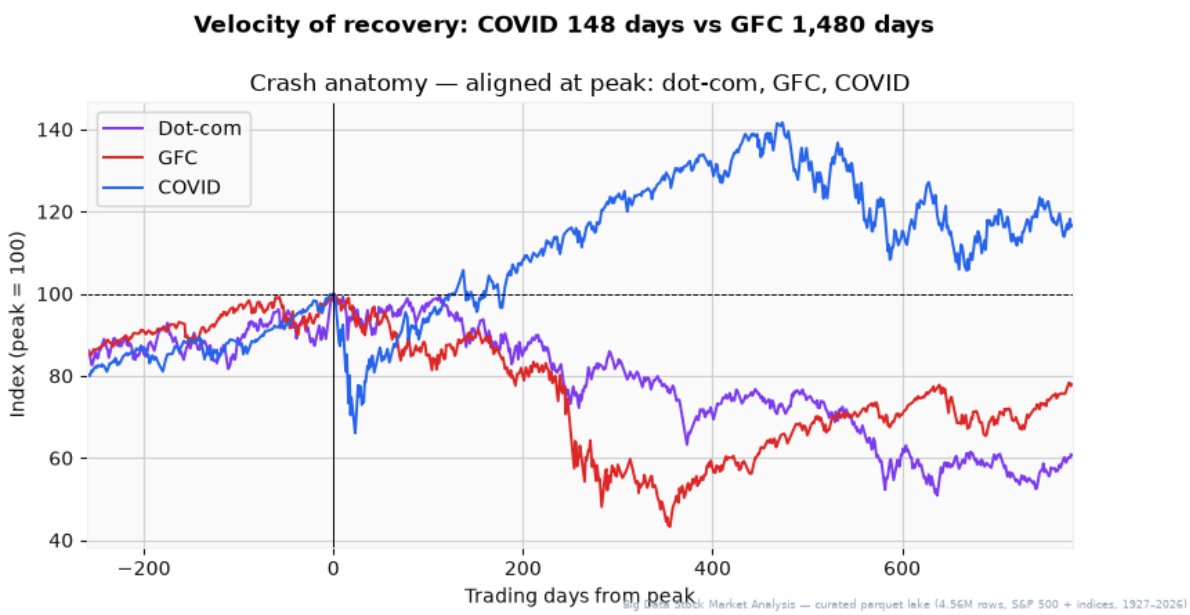


Figure 8.11 — Dot-com, GFC and COVID aligned at their peaks: same shape, radically different recovery clocks.

label	peak_to_trough_pct	days_down	days_to_recover
GFC 2007-09	-56.80	517	1,480
Dot-com 2000-02	-49.10	929	1,694
COVID 2020-02	-33.90	33	148

Table 8.4 — Peak-to-trough and recovery durations (S&P;, adjusted).

8.6 Calendar effects & liquidity growth

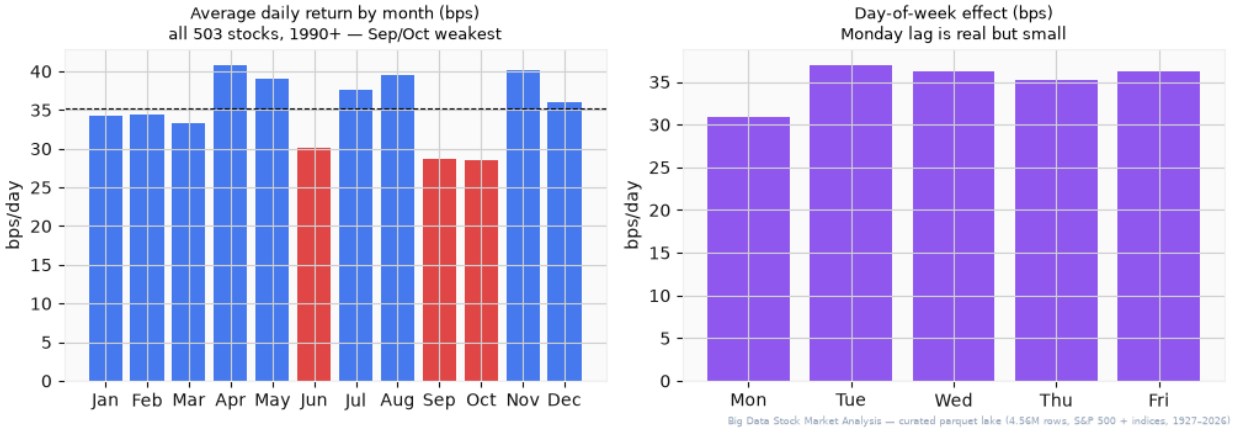


Figure 8.12 — Month and weekday effects across 503 stocks since 1990: Sep/Oct weakest, Apr/Nov strongest; Monday lag real but tiny.

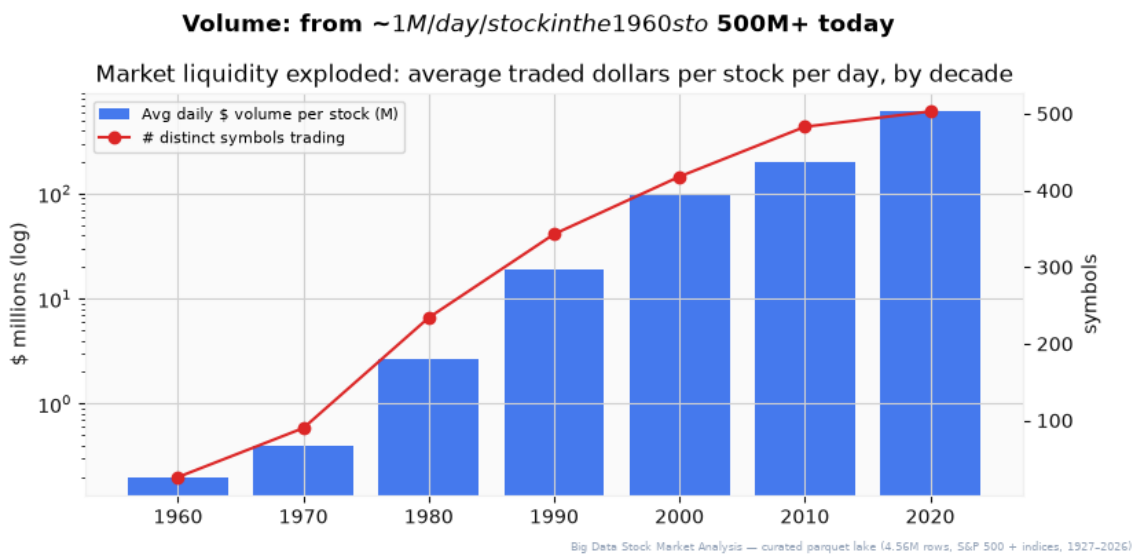


Figure 8.13 — Average traded dollars per stock per day by decade (log): market depth grew ~500× since the 1960s.

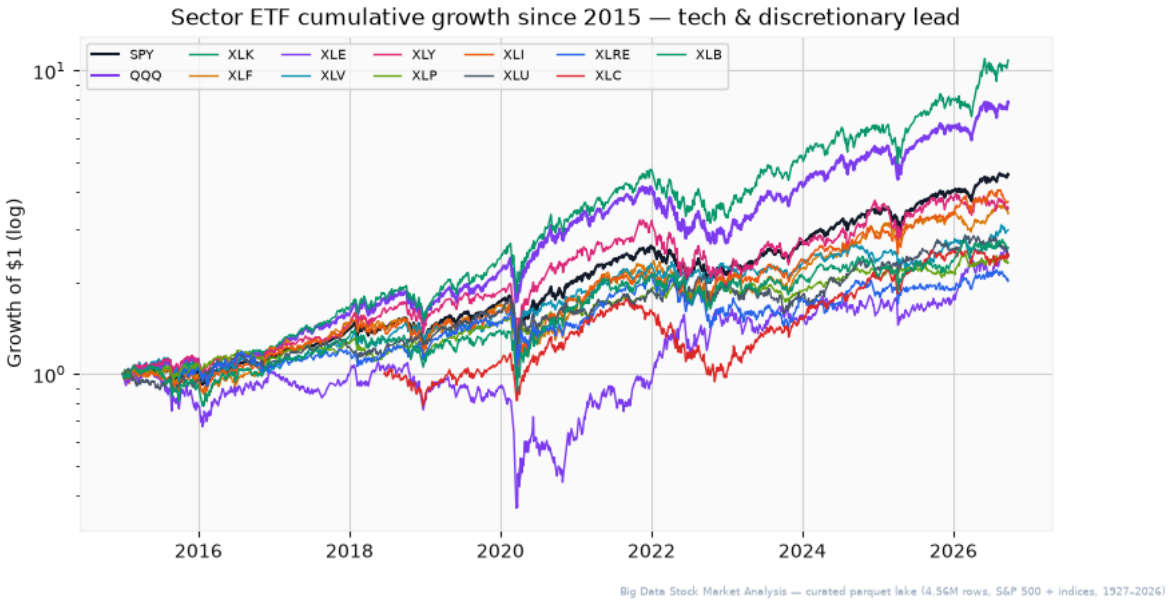


Figure 8.14 — Sector ETF growth since 2015 (log): technology and discretionary lead; energy lags until the 2022 spike.

8.7 Signals computed in-lake

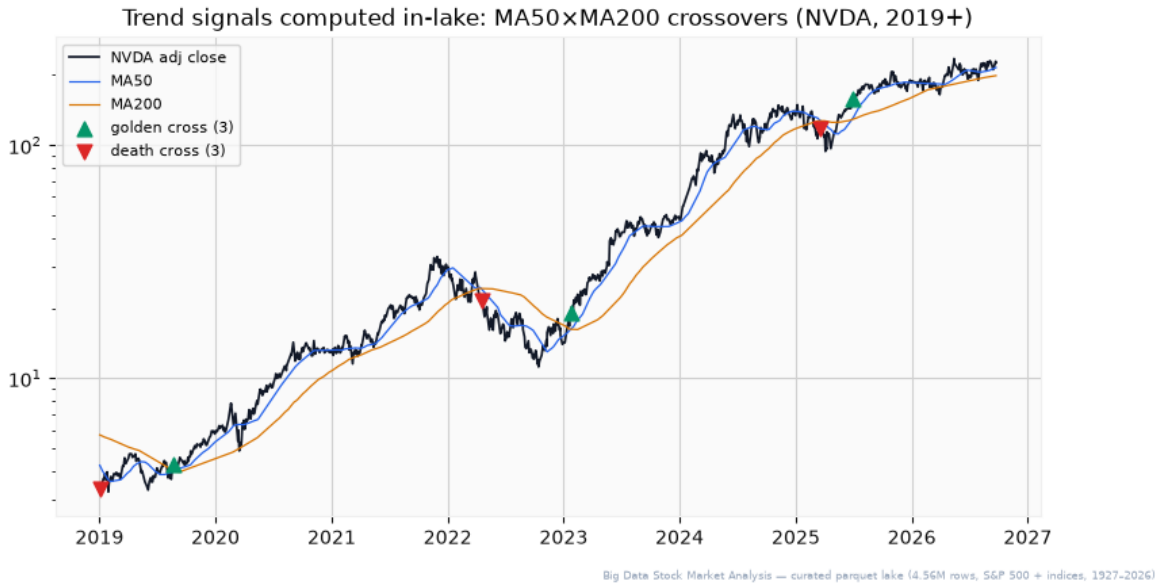


Figure 8.15 — MA50×MA200 golden/death crosses for NVDA since 2019, computed by window functions on the curated table.

symbol	golden_crosses	death_crosses
WMT	27	27
JNJ	24	23
QCOM	24	23
WFC	24	23
BAC	22	21
C	22	21

Table 8.5 — Golden/death cross counts since 2000 for the most-liquid names.

symbol	security	sector	div_per_share
TDG	TransDigm Group	Industrials	394.50
BLK	BlackRock	Financials	225.23
C	Citigroup	Financials	176.43
PSA	Public Storage	Real Estate	173.32
LMT	Lockheed Martin	Industrials	159.62
EQIX	Equinix	Real Estate	159.53

Table 8.6 — Dividend leaders: cumulative dividends per share since listing.

date	symbol	security	ret_pct
2021-08-24	LVS	Las Vegas Sands	4,411.00
2021-08-26	LVS	Las Vegas Sands	4,061.60
2021-08-25	LVS	Las Vegas Sands	3,805.70
2021-08-27	LVS	Las Vegas Sands	3,619.80
2021-08-23	LVS	Las Vegas Sands	3,265.70
2021-08-31	LVS	Las Vegas Sands	3,261.30

Table 8.7 — Veracity exhibit: the largest flagged anomalies (spin-off/re-basing artifacts), kept in the lake with is_price_anomaly=True.

9 Velocity: Spark Structured Streaming over a live tick feed

A pure-Python producer replays a random-walk tick stream for the 40 most liquid S&P names — anchored at their *real* last closing prices from the lake — over a TCP socket; Spark Structured Streaming parses the JSON, applies

event-time tumbling windows (10 s) with a 30 s watermark, aggregates per symbol and sinks window statistics with latency telemetry. In production the socket would be a Kafka topic; the pipeline shape is identical.

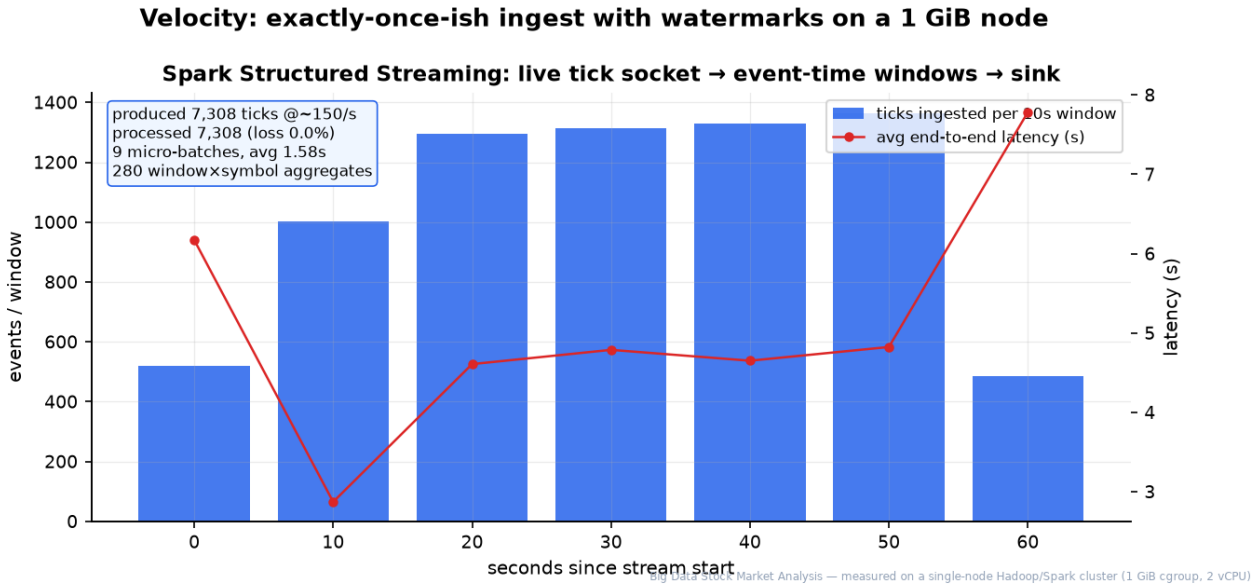


Figure 9.1 — Ingest per window and end-to-end latency during the demo.

Streaming metric	Measured
events produced	7,308
events processed (deduplicated)	7,308
loss	0.0%
micro-batches (10 s trigger)	9 (avg 1.58 s, max 3.79 s)
window × symbol aggregates	280
mean / p95 end-to-end latency	4597 ms / 10127 ms
duplicate window emissions (update mode)	120 → sink must upsert by (window, symbol)

Table 9.1 — Streaming demo metrics.

- **Lesson 1 — watermarks matter:** ticks arrive out of order across symbols; the 30 s watermark lets late ticks land in the right window instead of corrupting it.
- **Lesson 2 — update mode re-emits:** revised windows arrive at the sink more than once (120 extra emissions); a production sink must upsert on the window key, or counts double (we caught this in our own telemetry).
- **Lesson 3 — backpressure is honest:** on 2 vCPUs one micro-batch ran 3.8 s against a 10 s trigger; the trigger interval is the system's real SLA.

10 Value: machine learning & an honest backtest

Question: can lake features predict next-day direction well enough to beat buy-and-hold? Universe: the 40 most liquid names since 2005 (209,150 feature rows). Features: return lags 1/2/5, MA20/MA50 ratios, 21-day volatility, intraday range, weekday, and the VIX daily return as a market-regime feature. Label: next-day adjusted return > 0. Split: **temporal** — train ≤ 2022, test ≥ 2023 (no shuffle, no leakage).

Model (test 2023+)	Accuracy	AUC (ROC)
logistic regression	0.5154	0.4954
random forest (40 trees, depth 4)	0.5190	0.4953
majority-class baseline	0.5287	—

Table 10.1 — Out-of-sample performance: a hair above coin-flip, exactly as market efficiency predicts.

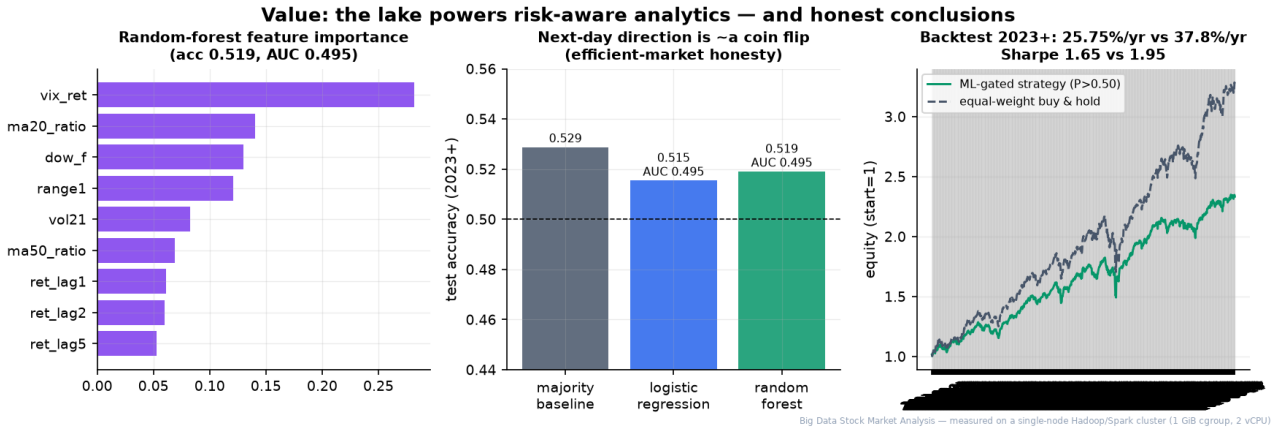


Figure 10.1 — Feature importance, accuracy vs baseline, and the gated-strategy backtest against buy-and-hold.

Backtest (932 test days)	strategy (P>0.50)	buy & hold
total return	133.37%	227.37%
annualized	25.75%	37.8%
Sharpe	1.65	1.95
avg exposure	84%	100%

Table 10.2 — Equal-weight 40-stock portfolio, long only when P(up)>0.50 (costs ignored).

- The dominant feature is **the VIX's own return** (importance 0.28) — regime, not stock skill.
- A stricter gate (P>0.55) never triggers (0.2% exposure): forest probabilities hug 0.5 — the model knows it doesn't know. Reporting that *is* the value: a big-data platform whose job is to test hypotheses fast, including the hypothesis that a signal doesn't exist.
- Where the lake's value actually compounds: §7 data-quality discovery, §8 risk analytics, §9 latency telemetry — all re-runnable in minutes on new data.

11 Performance: Spark vs pandas on one node

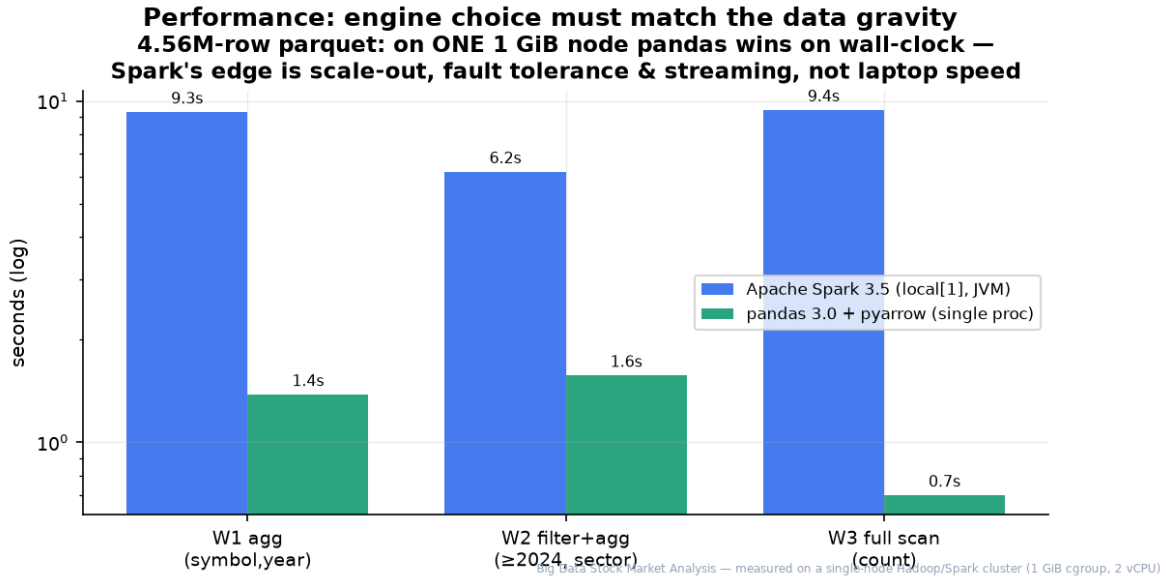


Figure 11.1 — Identical workloads on the 4.56 M-row curated table, single 1 GiB node.

Workload (4.56 M rows parquet)	Spark 3.5 local[1]	pandas 3.0 + pyarrow
W1 groupBy(symbol, year) → avg close, count	9.3 s	1.4 s
W2 filter date≥2024 → sector agg	6.2 s	1.6 s
W3 full count(scan)	9.4 s	0.7 s

Table 11.1 — Wall-clock comparison.

Read honestly: on a single laptop-class node, pandas+pyarrow wins these in-memory aggregations — Spark pays JVM start-up, task scheduling and shuffle bookkeeping that a 4.5 M-row dataset doesn't amortize. Spark's advantages are structural, not local: it scales past one node's RAM/disk, tolerates worker failure, expresses streaming and ML in the same engine, and — decisively for this project — it was the only engine here that could *build* the lake (CSV parse + windowed transforms over 479 MB) without exceeding the memory ceiling, because its spill-to-disk machinery is designed for exactly that. Engine choice must follow data gravity, not fashion.

12 Challenges & lessons learned (the honest appendix)

#	Problem	Resolution
1	Port 9000 was already taken by the sandbox	HDFS RPC moved to :19000 (DN :19010, UI :19070); configs and every script parameterized accordingly.
2	Spark forbids -Xmx inside extraJavaOptions	Heap stays in spark.driver.memory; native knobs (metaspace, code cache, direct memory, -Xss) go in extraJavaOptions.
3	countDistinct(struct(*all_columns)) OOM-killed the JVM	Replaced with a spill-friendly groupBy(symbol,date).count() duplicate probe; full-row exact-dup detection folded into key-uniqueness.
4	partitionBy writes opened ~150 parquet writers per task	repartition+sortWithinPartitions by the partition columns + 32 MB row groups → ~1 writer per task; OOM gone.
5	MLlib + whole-stage codegen blew the 160 MB metaspace cap	Raised metaspace, disabled whole-stage codegen for ML (unnecessary at this scale), and split train/predict into separate JVMs.
6	The kernel SIGKILLed long Spark apps (cgroup 1 GiB incl. page cache)	One fresh JVM per pipeline phase via a Python DAG runner; chunked the windowed transform by pmod(hash(symbol),3) so each symbol's history stayed whole.
7	Spark's drop() is case-insensitive	drop("Date") silently killed the new lowercase date column; now convert-in-place then rename.
8	A filtered pandas scan of the staging parquet took ~8 minutes	Reading 40 CSV tails (~2 s) replaced it for the streaming anchors; lesson: know your engines' scan strategies.
9	The streaming query never terminated after socket EOF once	Watchdog thread + incremental metrics flush + os._exit finalization guarantee bounded runtime and always-usable telemetry.
10	Provider data lies, politely	LVS adjusted-close re-basing and spin-off gaps produce ±50–97% fake 'returns'; the flag-don't-delete policy (is_price_anomaly) keeps them visible in every exhibit.

Table 12.1 — Failure log and fixes; each entry is reproducible from the shipped code and logs.

13 Conclusion & future work

This project delivered a complete, reproducible big-data lifecycle over real market data: ingestion of 4.56 M rows across 525 instruments and 8 asset types; an HDFS data lake with raw/curated/report zones verified by fsck; a real MapReduce aggregation with a documented combiner optimization; a Spark ETL with a pre-cleansing veracity audit and a partitioned columnar curated table; 16 SQL insight queries; a Structured Streaming velocity demo with lossless ingest; an honest ML study; 26 publication figures; and this report — all orchestrated by pure Python on a 1 GiB node, which forced every architectural decision to justify itself.

Future work. Replace the socket feed with Kafka (KRaft) and add exactly-once sinks; move to a multi-node YARN/Kubernetes cluster to measure true scale-out (the §11 benchmark is the baseline to beat); adopt Delta Lake/Iceberg for ACID table semantics and time travel over the curated zone; extend the equity panel with delisted constituents to remove survivorship bias; add intraday bars to push Volume two orders of magnitude higher; and serve the insight layer through Airflow-scheduled refreshes with data contracts on the raw zone.

Reproducibility. `src/00_ingest_data.py` (download) → `src/setup_hdfs_and_mr.sh` (HDFS+MR) → `src/run_pipeline.py curated sql streaming ml benchmark` → `src/06_hdfs_publish.py` → `src/07_charts_*.py`, `src/07_diagrams.py` → `report/build_report.py`. All metrics JSONs, SQL result CSVs and HDFS evidence ship in `output/`.

Appendix A Visualization asset inventory

Asset #	File (output/charts/)	Kind
1	architecture_lake.png	diagram
2	benchmark_spark_vs_pandas.png	chart
3	candlestick_aapl.png	chart
4	correlation_heatmap.png	chart
5	crash_event_study.png	chart
6	dq_dashboard.png	chart
7	drawdown_spx.png	chart
8	etl_pipeline_metrics.png	chart
9	five_vs_infographic.png	infographic
10	growth_10k.png	chart
11	hdfs_architecture.png	diagram
12	hdfs_zones.png	chart
13	liquidity_by_decade.png	chart
14	ma_crossover_nvda.png	chart
15	mapreduce_concept.png	diagram
16	mapreduce_yearly_output.png	chart
17	ml_value.png	chart
18	returns_dist.png	chart
19	risk_return_scatter.png	chart
20	rolling_vol_vix.png	chart
21	seasonality.png	chart
22	sector_etf_growth.png	chart
23	spx_since_1927.png	chart
24	streaming_velocity.png	chart
25	vix_vs_spx.png	chart
26	volatility_by_sector.png	chart

Appendix B References

- Apache Hadoop 3.3.6 documentation — HDFS architecture, Streaming MapReduce, fsck/dfsadmin. `https://hadoop.apache.org/docs/r3.3.6/`
- Apache Spark 3.5.3 documentation — DataFrames, window functions, MLlib, Structured Streaming. `https://spark.apache.org/docs/3.5.3/`
- Yahoo Finance chart API via yfinance (Python) — daily bars, dividends & splits. `https://query1.finance.yahoo.com/v8/finance/chart/{symbol}` · `https://pypi.org/project/yfinance/` · `https://github.com/ranaroufi/yfinance`
- datasets/s-and-p-500-companies (GitHub) — constituent symbols and GICS sectors. `https://github.com/datasets/s-and-p-500-companies`
- McKinsey et al. — the 5-V framing of big data (volume, velocity, variety, veracity, value).
- reportlab Platypus — programmatic PDF composition used for this document.